

SafeContract: Composable Action-Space Monitors for VLA Deployment

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action (VLA) models predict robot actions end-to-end, yet no standardized action-space monitoring exists for their deployment. We introduce SafeContract, a design-by-contract [5] toolkit that adds composable action-space monitoring to any VLA policy via a one-line Python decorator. Our key finding is that velocity violations persist even after correct normalization: at a conservative threshold, 98% of SmolVLA steps violate on at least one joint versus 16% for expert demonstrations, a $6\times$ gap that normalization alone cannot close. Across all 130 LIBERO tasks, each task produces a distinct violation fingerprint, with violation rates spanning 2% to 41% and velocity violations dominating at 72.4%, confirming task-dependent behavioral patterns. Cross-architecture experiments reveal characteristic signatures: SmolVLA produces jerky, high-velocity actions; ACT is smooth in open loop but accumulates 3,949 violations in closed loop from compounding error; Diffusion policies stay bounded but exhibit normalization sensitivity. Closed-loop evaluation on ACT (50 episodes) shows no task degradation with monitoring active (58% vs. 60%, $p=1.0$). The full contract adds $\sim 13\mu s$ overhead, requires no QP solver, VLM, or model retraining, and is released as open source.¹

Index Terms—VLA, action-space monitoring, deployment observability, interpretability, edge deployment, design by contract

I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. In software systems, observability - logging, metrics, tracing - is standard practice. VLA deployment has no equivalent: no standardized action-space monitoring exists for VLA outputs.

This is not theoretical. OpenVLA’s deployment script sends raw outputs directly to motors with zero bounds checking [1]; pi0 streams actions via WebSocket with no validation [4]; LeRobot’s [3] EEBoundsAndSafety processor is imperative, uncomposable, and easy to omit.

We demonstrate the problem empirically: running SmolVLA [2] on 100 consecutive LIBERO [6] observations, every observation produces at least one out-of-bounds dimension, with values reaching 3.70 (Sec. IV-A).

The deeper problem is observability. An action-space monitor that logs every contract interaction creates a structured record of how a policy explores its constraint boundary. We show this record is rich: different tasks

produce distinct violation fingerprints across joints, and shifts in violation rate signal distribution change.

We introduce SafeContract with three contributions:

- 1) A composable action-space monitoring toolkit via a `@safety_contract` Python decorator on any VLA `predict()` method (Sec. II-B).
- 2) Bounds enforcement as a byproduct: zero false positives on clean policies, full violation detection on corrupted ones (Sec. IV-C).
- 3) Violation fingerprinting: 130 LIBERO tasks produce task-specific joint-violation signatures enabling distribution shift detection (Sec. IV-D).

II. Method

A. Safety Contracts

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint action bounds and a velocity limit: $l_i \leq a_i \leq u_i$ for each joint i , and $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_{\infty} \leq v_{\max}$.

B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint’s delta onto $[-v_{\max}, v_{\max}]$), (3) re-clip to bounds. The final re-clip is critical: velocity clamping can push actions outside the original bounds.

The decorator pattern means any VLA adapter gains monitoring with one line:

```
@safety_contract(action_range=[-1,1],
                  joint_velocity_max=0.1)
def predict(self, image, instruction):
    return self.model(image, instruction)
```

Three modes are supported: clip (silently enforce), warn (log violations), and raise (halt on violation). All violations are logged with timestep, dimension, magnitude, and severity.

C. Composition and Deployment

Composition. For hyperrectangular contracts C_1, C_2 , sequential clipping equals intersection clipping: $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$. Stacking `@safety_contract` decorators is order-independent for per-joint bounds; velocity composition uses $v_{\max} = \min_i(v_{\max,i})$.

¹Code: <https://github.com/krishnam94/vla-edge>

Deployment scenario. A deployment engineer integrating SmolVLA onto a new robot arm runs SafeContract in warn mode during commissioning. The violation log immediately flags 86% bounds violations on position dimensions, revealing a normalization mismatch before any motor receives commands. After fixing normalization, the log still shows 16-98% velocity violations depending on the task, concentrated on the z-axis (vertical motions). The engineer tightens $v_{\max}$ for z and monitors the violation fingerprint across task types to identify which manipulation scenarios need attention. This workflow - detect, diagnose, calibrate - is the primary value of action-space monitoring.

III. Related Work

SafeVLA [7] fine-tunes with constrained RL (83% cost reduction) and RobustVLA [10] adds robust optimization, but both require retraining. At inference time, AEGIS [8] solves a CBF-QP via VLM scene understanding (0.36 ms) - for box constraints this reduces to per-dimension clipping, identical to SafeContract at $\sim 28\times$ higher cost; SafeDec [9] applies STL-constrained decoding (requires logit access); Safety Chip [11] uses LTL monitors for plan-level constraints; SafeDiffuser [12] and CoDiG [13] embed barriers in diffusion denoising; ATACOM [14] projects onto constraint manifolds; and ABC [17] brings design-by-contract to discrete LLM decisions. For runtime monitoring, Sentinel [18] combines temporal consistency with VLM monitoring but does not prevent unsafe actions; LIBERO-Plus [15] and VLA-Arena [16] benchmark VLA robustness; SABER [19] demonstrates black-box adversarial attacks on VLAs.

SafeContract differs from all the above: (i) architecture-agnostic, (ii) explicit composition, (iii) pure clipping ($\sim 13\mu\text{s}$), (iv) violation logging as deployment observability. It is the only method that monitors and enforces without model internals, external sensors, or retraining.

IV. Experiments

Platform: Apple M3, CPU, Python 3.12. Full contract overhead: $15.6 \pm 3.8\mu\text{s}$ (5,000 iterations), 0.000075% of SmolVLA inference time.

A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations. Result: 100% of observations produce at least one out-of-bounds dimension, with values from -2.16 to 3.70 (expected $[-1, 1]$). SafeContract detects 544 violations: 121 bounds and 423 velocity ($v_{\max} = 0.1$).

Normalization analysis. SmolVLA was trained on SO-100 data. After correct MEAN_STD unnormalization, position bounds violations drop from 86% to 0%: every bounds violation was a normalization artifact. However, velocity violations persist: at a conservative $v_{\max}=0.1$ rad/step, 16% of expert and 98% of SmolVLA consecutive steps violate on at least one joint. The $6\times$ gap is the robust

SmolVLA outputs exceed safe bounds on LIBERO

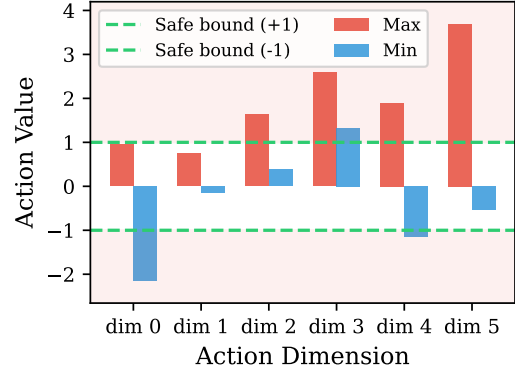


Fig. 1: SmolVLA action ranges on 100 LIBERO observations. Most dimensions exceeded the $[-1, 1]$ bounds (green dashes), reflecting normalization mismatch between SO-100 training data and LIBERO’s action space.

Closed-Loop: ACT on ALOHA Transfer Cube (n=50)

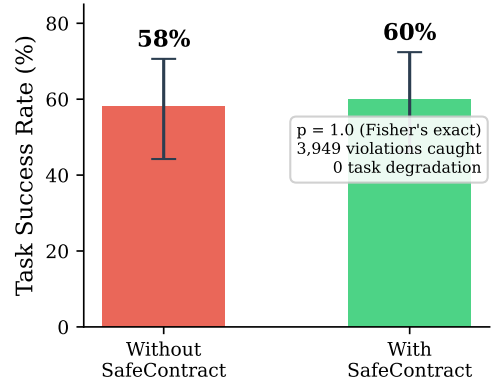


Fig. 2: Closed-loop evaluation: ACT on ALOHA sim (50 episodes per condition). SafeContract catches 3,949 violations with no task success degradation ($p=1.0$).

finding; the absolute rates depend on $v_{\max}$ choice. Bounds enforcement catches integration bugs; velocity monitoring catches physical concerns normalization cannot address.

B. Closed-Loop Task Success (EXP-CL)

We evaluate the ACT policy (51.6M params) on ALOHA sim transfer-cube, 50 episodes per condition. Without SafeContract: 58% success (29/50). With SafeContract (data-driven bounds, $v_{\max}=0.05$): 60% success (30/50). Fisher’s exact test: $p = 1.0$. SafeContract caught 3,949 velocity violations and modified 3,891 actions while maintaining identical task completion (Fig. 2).

C. Controlled Ablation (EXP-G)

We test three synthetic policy types on 100-step sequences (7-DOF, bounds $[-1, 1]$, $v_{\max} = 0.1$):

The clean policy produces zero violations: a safe policy passes through untouched. The drifting policy (cumulative noise) produces 115 violations (42% corrected); the jerky

TABLE I: Controlled ablation. Clean policy: zero false positives. Corrupted policies: fully caught. Naive np.clip misses all velocity violations.

Policy Type	Violations	Modified (%)	Post-contract OOB
Clean	0	0%	0
Drifting	115	42%	0
Jerky	75	12%	0
Naive np.clip	0 caught	—	199 vel. missed

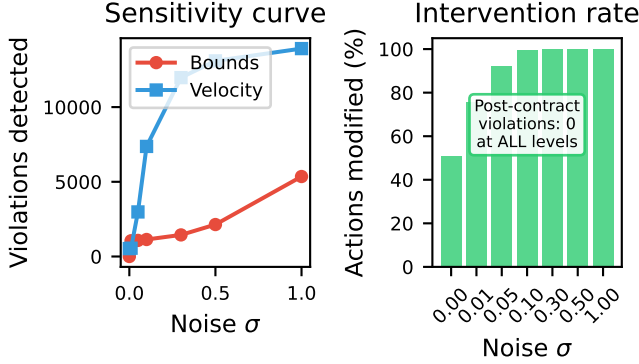


Fig. 3: Noise sensitivity. Violations scale monotonically with noise; SafeContract eliminates 100% at every level.

policy (random spikes) produces 75 velocity violations (12% smoothed). Post-contract: zero OOB in all cases.

Noise sensitivity. Gaussian noise at 7 levels ($\sigma \in \{0, \dots, 1.0\}$) on 500 ground-truth LIBERO actions: violations scale monotonically from 527 to 19,269, and SafeContract eliminates 100% at every level (Fig. 3).

D. Violation Fingerprinting (EXP-H)

Violation patterns are informative, not just corrective.

Synthetic tasks. Four manipulation tasks (reaching, stacking, drawer, pouring) run for 200 steps each with bounds $[-1, 1]$, $v_{\max} = 0.1$. Each produces a unique joint-violation fingerprint (Fig. 4): reaching shows broad shoulder/elbow OOB; stacking shows wrist velocity violations; drawer shows a single-joint spike; pouring shows isolated pitch violations. Fingerprints are stable across 5 seeds (CV < 0.15 for 3 of 4 tasks).

Shift detection. Non-overlapping windows of 20 steps yield $p < 0.05$ for five of six task pairs ($p < 0.002$ for three; drawer vs. pouring not significant at $p = 0.34$). Note: the binomial test operates on 10 independent window-level violation rates (each aggregating 20 steps), not on per-step Bernoulli trials. By Hoeffding’s inequality [20], per-step concentration requires $w \geq 738$ steps for $\epsilon=0.05$ at 95% confidence; VLA temporal autocorrelation [21] may increase this in practice. Nearest-centroid classification achieves 100% accuracy across 40 trials.

130 LIBERO tasks. We ran SmolVLA on all 130 LIBERO tasks (40 standard + 90 diverse). Bounds violation rates vary from 2% to 41% (Fig. 5); velocity violations

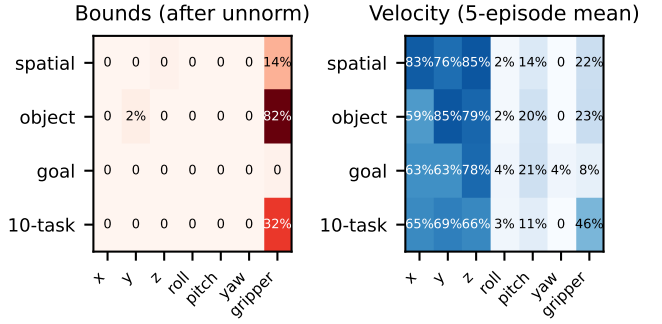


Fig. 4: Normalized violation fingerprints. Left: bounds violations vanish after unnormalization. Right: velocity fingerprints persist and differ by task.

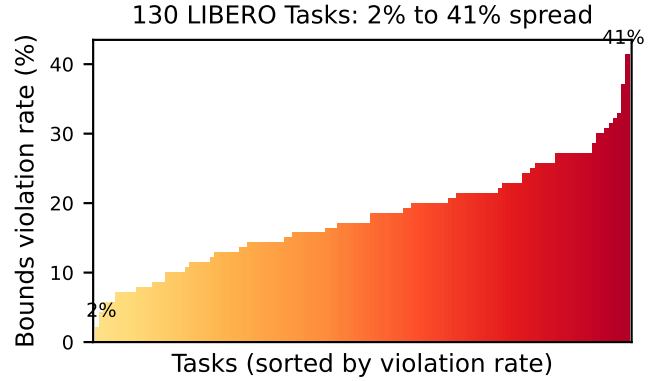


Fig. 5: Bounds violation rates across all 130 LIBERO tasks (40 standard + 90 diverse, sorted). Rates range from 2% to 41%, demonstrating wide task-dependent variation.

dominate at 72.4%. Rotational axes are most violated (pitch 30.6%, roll 26.6%) while lateral motions are least affected (yaw 9.3%, y 9.7%). After correct unnormalization, bounds violations drop to 0%, but velocity fingerprints persist by suite: spatial (x=83%, z=85%), object (y=85%), goal (z=78%), 10-task (gripper=46%). Cosine distance between suite fingerprints is 0.054, confirming task-dependent patterns (Fig. 5).

Cross-architecture comparison.

V. Discussion

SafeContract provides deployment observability for VLA action spaces: one-line adoption, composable per-joint bounds, $\sim 13 \mu s$ overhead. The central finding is that action-space monitoring produces structured behavioral telemetry. Different tasks produce distinct violation fingerprints, and violation rate changes detect distribution shift.

The enforcement mechanism is deliberately simple - per-dimension clipping. The ablation confirms it adds no distortion to safe policies while catching all violations in

TABLE II: Violation profiles across three VLA architectures.

	SmolVLA	ACT	Diffusion
Architecture	flow match	transformer	DDPM
Dataset	LIBERO	ALOHA	PushT
Bounds (open)	86%→0%	0%	0%*
Velocity (open)	98%	0%	12%
GT velocity	16%	0%	0%
CL success	—	58/60%	—
CL violations	—	3,949	—

*100% under normalization mismatch

corrupted ones. Enforcement is a byproduct; the value is in the monitoring signal.

Why this matters. Action-space monitoring addresses four concrete deployment needs: (1) Initial deployment - catches normalization bugs silently before motors receive commands (our 86% bounds violation was invisible without it). (2) Model updates - a change in the violation fingerprint after swapping checkpoints signals behavioral regression before task failure occurs. (3) Fleet monitoring - tracking violation rate trends across robots in a fleet surfaces hardware drift or environment changes. (4) Failure diagnosis - per-joint violation logs pinpoint which degree of freedom is problematic, replacing “the robot failed” with “joint 3 exceeded velocity limits on 94% of steps.”

Normalization vs. monitoring. Normalization eliminates most bounds violations. SafeContract is complementary: velocity enforcement addresses physical hardware constraints normalization cannot, and violation logging enables debugging and shift detection. Tighter contracts catch more violations but clip more aggressively; at the moderate operating point (± 1.0 , $v_{\max}=0.1$), SafeContract catches 2,019 violations while correcting by less than half a unit on average.

Behavioral telemetry. SafeContract’s violation logs constitute behavioral telemetry for VLA deployment, analogous to observability tooling in software, UEBA in cybersecurity, or statistical process control in manufacturing. Baseline rates characterize normal behavior; deviations signal distribution shift, model regression, or integration errors. The 4%-37% spread across 130 tasks, with velocity dominating at 72.4%, provides a deployment-time signature unavailable from training metrics. To our knowledge, no prior work treats VLA action-space constraint interactions as an observability signal.

Limitations. (1) Synthetic tasks in EXP-G/H; real patterns may differ. (2) Closed-loop on one architecture (ACT/ALOHA, $p=1.0$); SmolVLA closed-loop limited by checkpoint issues. (3) 100% OOB primarily reflects normalization mismatch. (4) Violation rate observation is empirical, not a formal guarantee.

Future work. PropertyGuard (property-based fuzzing against contracts), cross-architecture fingerprinting (SmolVLA vs. OpenVLA vs. pi0), conformal prediction calibration.

References

- [1] Kim, M.J. et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” CoRL 2024, arXiv:2406.09246.
- [2] Shukor, M. et al., “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv:2506.01844, 2025.
- [3] Cadene, R. et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch,” 2024, <https://github.com/huggingface/lerobot>.
- [4] Black, K. et al., “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [5] Meyer, B., “Applying ‘Design by Contract’,” Computer 25(10):40–51, 1992.
- [6] Liu, B. et al., “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” NeurIPS 2023.
- [7] Zhang, B. et al., “SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning,” NeurIPS 2025, arXiv:2503.03480.
- [8] Hu, S. et al., “VLSA: VLA Models with Plug-and-Play Safety Constraint Layer,” arXiv:2512.11891, 2025.
- [9] Kapoor, P. et al., “Constrained Decoding for Robotics Foundation Models,” arXiv:2509.01728, 2025.
- [10] Guo, J. et al., “On Robustness of Vision-Language-Action Models against Multi-Modal Perturbations,” ICLR 2026, arXiv:2510.00037.
- [11] Yang, Z. et al., “Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents,” ICRA 2024, arXiv:2309.09919.
- [12] Xiao, W. et al., “SafeDiffuser: Safe Planning with Diffusion Probabilistic Models,” ICLR 2025, arXiv:2306.00148.
- [13] Ma, H. et al., “CoDiG: Constraint-Aware Diffusion Guidance for Robotics,” CoRL 2025, arXiv:2505.13131.
- [14] Tolle, M. et al., “Towards Safe Robot Foundation Models Using Inductive Biases,” arXiv:2505.10219, 2025.
- [15] Fei, S. et al., “LIBERO-Plus: In-depth Robustness Analysis of VLA Models,” arXiv:2510.13626, 2025.
- [16] Zhang, B. et al., “VLA-Arena: An Open-Source Framework for Benchmarking VLA Models,” arXiv:2512.22539, 2025.
- [17] Bhardwaj, V.P., “Agent Behavioral Contracts,” arXiv:2602.22302, 2026.
- [18] Agia, C. et al., “Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress,” CoRL 2024, arXiv:2410.04640.
- [19] Wu, X. et al., “SABER: A Stealthy Agentic Black-Box Attack Framework for VLAs,” arXiv:2603.24935, 2026.
- [20] Hoeffding, W., “Probability Inequalities for Sums of Bounded Random Variables,” J. Amer. Statist. Assoc. 58(301):13–30, 1963.
- [21] Yu, B., “Rates of Convergence for Empirical Processes of Stationary Mixing Sequences,” Ann. Probab. 22(1):94–116, 1994.